

Recap



- MPI programs
- Socket API function – send / recv / close
- Processor Synchrony
- Synchronous / Asynchronous executions
- Virtual synchrony
- Mutual Exclusion

IIT ROORKEE ■ ■ ■

Quorum based mutual exclusion

- Permission is obtained from only a subset of other processes, called the **Request Set (or Quorum)**
- Separate Request Set S_i for each process i
- Requirements:
 - (a) for all $i, j: i \neq j :: S_i \cap S_j \neq \Phi$
 - (b) for all $i: i \in S_i$
 - (c) for all $i: |S_i| = K$, for some K
 - (d) any node i is contained in exactly D Request Sets, for some D
- Finite projective plane of N points

Degenerated set of S_i 's

- First create a symmetric set of S_i 's for M where
 - $(M - 1)$ is a power of a prime number and
 - M is the smallest integer larger than L .
- In this set of S_i 's for $K = M$, each component is contained in M subsets
- Now replace each component greater than $N = L(L - 1) + 1$ in this set of S_i 's by a number smaller than or equal to N
- This replacement is made by a different number each time.
- The resulting set of S_i 's is not symmetric in the sense that the size of S_i is not always L .
- Degenerated sets can be created in the similar way if $N \neq K(K - 1) + 1$

- To obtain S_i 's for $N = 5$, for instance, first create a set of S_i 's for 7 and then replace 7 and 6 with 5 and 4, respectively, and remove S_7 and S_6

$$S_1 = \{1, 2, 3\}$$

$$S_4 = \{1, 4, 5\}$$

~~$$S_6 = \{1, 6, 7\}$$~~

$$S_2 = \{2, 4, 6\}$$

$$S_5 = \{2, 5, 7\}$$

~~$$S_7 = \{3, 4, 7\}$$~~

$$S_3 = \{3, 5, 6\}$$

$$S1 = \{1, 2, 3\}$$

$$S2 = \{2, 4\}$$

$$S3 = \{3, 4, 5\}$$

$$S4 = \{1, 4, 5\}$$

$$S5 = \{2, 5\}$$

Method 2: Use a grid of $L \times L$

Maekawa's Algorithm

- To request critical section:
 - i sends REQUEST message to all process in S_i
(The seq no of the REQUEST message > Seq no of any REQUEST message sent / received / observed at this node)
- On receiving a REQUEST message:
 - send a REPLY message if no REPLY message has been sent since the last RELEASE message it received.
Update status to indicate that a REPLY has been sent.
 - Otherwise, queue up the REQUEST (ordered queue)

- To enter critical section:
 - i enters critical section after receiving REPLY from all nodes in S_i
- To release critical section:
 - send RELEASE message to all nodes in S_i
- On receiving a RELEASE message,
 - send REPLY to next node in queue and delete the node from the queue.
 - If queue is empty, **update status to indicate no REPLY message has been sent.**

- Maekawa's algorithm achieves mutual exclusion
Assume both S_i and S_j are in CS concurrently
Let k be the site in both S_i and S_j
Both i and j are in CS $\Rightarrow$ k has given permission to both (not possible)
- Message Complexity: $3 \cdot \sqrt{N}$
- Synchronization delay =
 $2 \cdot (\text{max message transmission time})$
- Major problem: DEADLOCK possible

DEADLOCK

Nodes requesting for CS: 1, 2, 4 and 6

$$S_1 = \{1, 2, 3\}$$

$$S_4 = \{1, 4, 5\}$$

$$S_6 = \{1, 6, 7\}$$

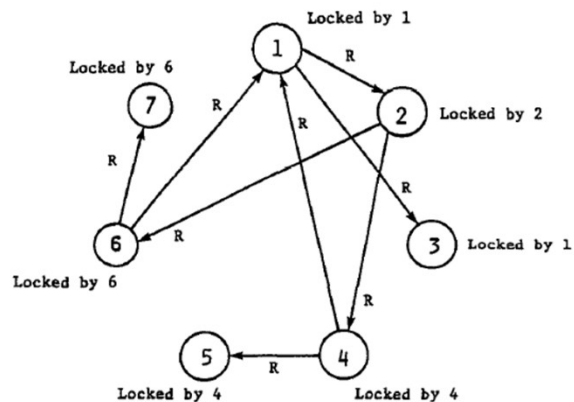
$$S_2 = \{2, 4, 6\}$$

$$S_5 = \{2, 5, 7\}$$

$$S_7 = \{3, 4, 7\}$$

$$S_3 = \{3, 5, 6\}$$

(b)



Circular wait

Need three more types of messages (FAILED, INQUIRE, YIELD) to handle deadlock

FAILED: A FAILED message from site S_i to site S_j indicates that S_i can not grant S_j 's request because it has currently granted permission to a site with a higher priority request.

INQUIRE: An INQUIRE message from S_i to S_j indicates that S_i would like to find out from S_j if it has succeeded in locking all the sites in its request set.

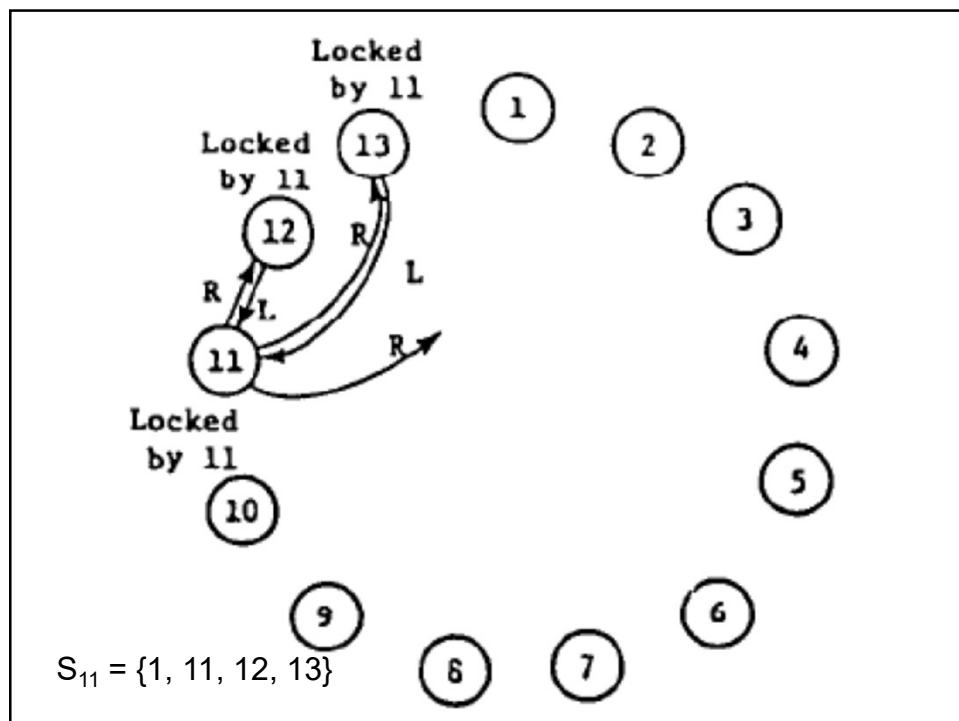
YIELD: A YIELD message from site S_i to S_j indicates that S_i is returning the permission to S_j

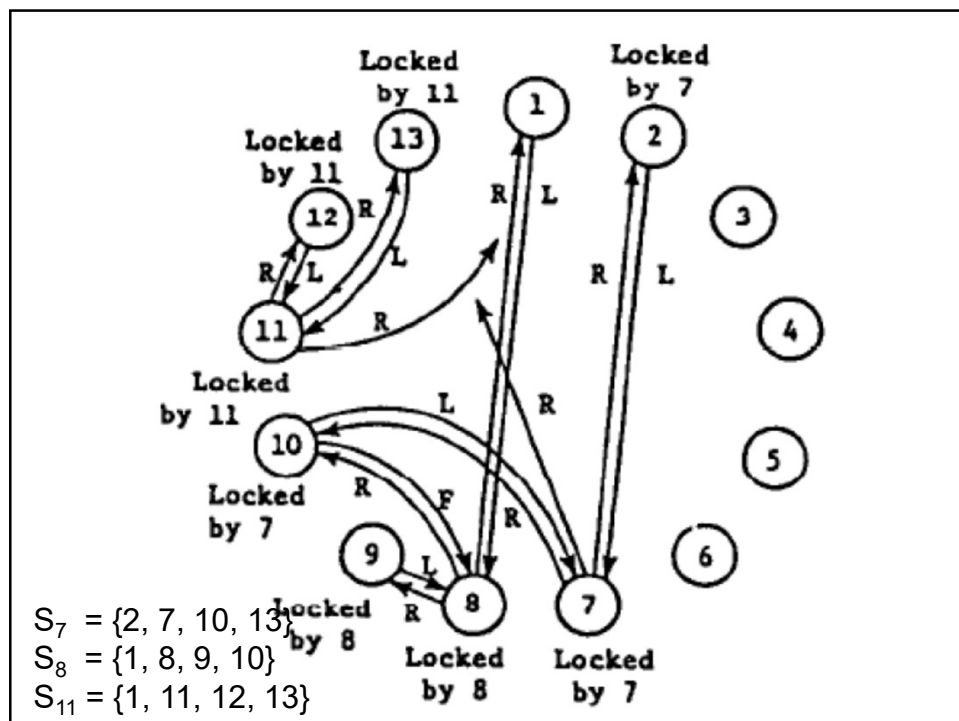
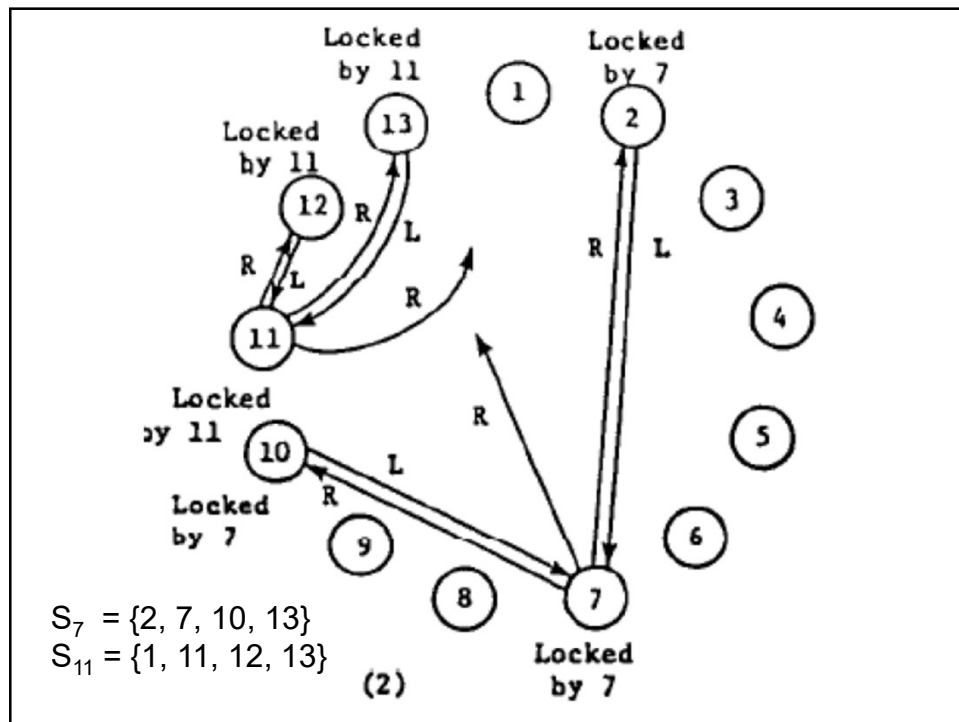
Maekawa's algorithm handles deadlocks as follows:

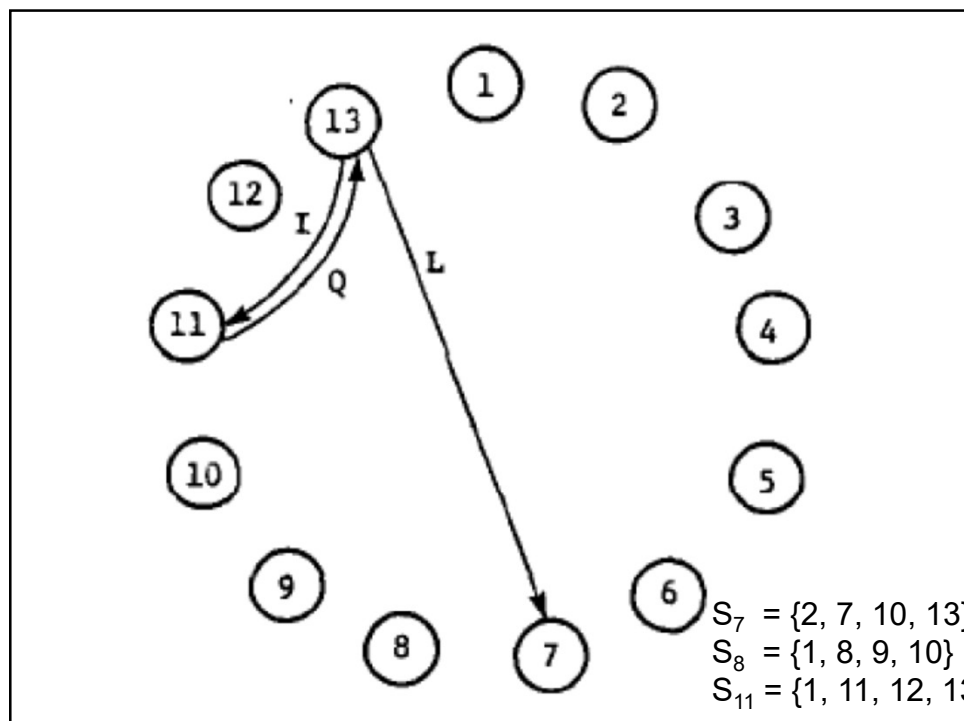
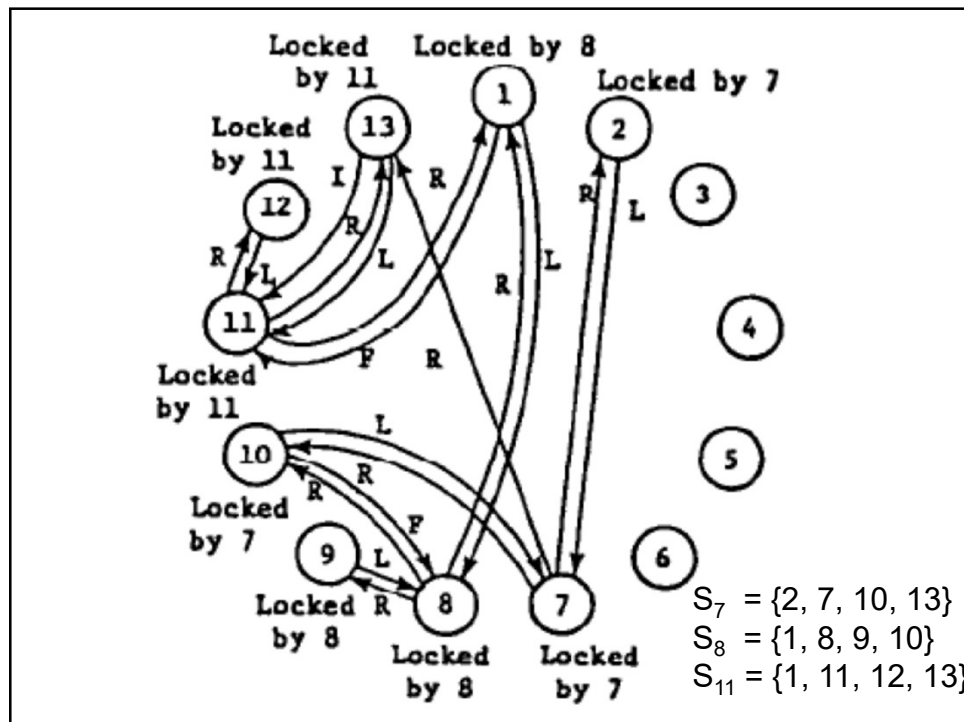
1. When a REQUEST(ts, i) from site S_i blocks at site S_j (placed in queue) because S_j has currently granted permission to site S_k , **then**
 - S_j sends a FAILED(j) message to S_i if S_i 's request has lower priority (higher timestamp).
 - **Else**, S_j sends an INQUIRE(j) message to site S_k .

1. In response to an INQUIRE(j) message from site S_j , site S_k sends a YIELD(k) message to S_j provided
 - S_k has received a FAILED message from a site in its request set or
 - if it has sent a YIELD to any of these sites, but has not received a new REPLY from it.
2. In response to a YIELD(k) message from site S_k , site S_j assumes as if it has been released by S_k ,
 - places the request of S_k at appropriate location in the request queue, and
 - sends a REPLY(j) to the top request's site in the queue.

S_1	=	{1.	2.	3.	4}
S_5	=	{1.	5.	6.	7}
S_8	=	{1.	8.	9.	10}
S_{11}	=	{1.	11.	12.	13}
S_2	=	{2.	5.	8.	11}
S_6	=	{2.	6.	9.	12}
S_7	=	{2.	7.	10.	13}
S_{10}	=	{3.	5.	10.	12}
S_3	=	{3.	6.	8.	13}
S_9	=	{3.	7.	9.	11}
S_{13}	=	{4.	5.	9.	13}
S_4	=	{4.	6.	10.	11}
S_{12}	=	{4.	7.	8.	12}







Maekawa's Algorithm



- Maekawa's algorithm requires 3 extra messages to handle deadlocks
- Message complexity can be up to $5 \cdot \sqrt{N}$
 - Light Load
 - Heavy Load
- Building the request sets?
- Node Failure (fail-stop faults only)
 - Some node detects the fault and takes over the role of the failed node – degenerated sets

Agarwal-El Abbadi Quorum-Based Algorithm



- Message complexity
 - Lamport's Algorithm
 - Ricart-Agarwala Algorithm
 - Thomas Algorithm (majority consensus rule)
 - Maekawa's Algorithm
 - **Agarwal-El Abbadi Algorithm**
- Sites may either crash or may fail to send or receive messages
- Fault Tolerance in case of above algorithms

Agarwal-El Abbadi Quorum-Based Algorithm



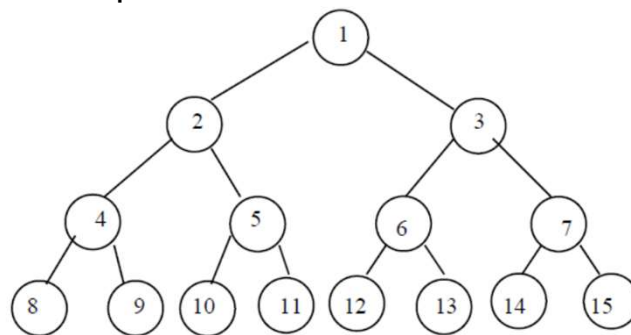
- Uses tree-structured quorums
- All the sites in the system are logically organized into a complete binary tree
- For a complete binary tree with level k , root is at level k and leaves are at level 0
- The number of sites in a path from root to a leaf is equal to the level of the tree $= k + 1 = O(\log n)$
- A path in a binary tree is the sequence $a_1, a_2, \dots, a_i, a_{i+1}, \dots, a_k$ such that a_i is the parent of a_{i+1} .

IIT ROORKEE

21

Quorums

When there is no node failure, the number of quorums formed is equal to the number of leaf sites.



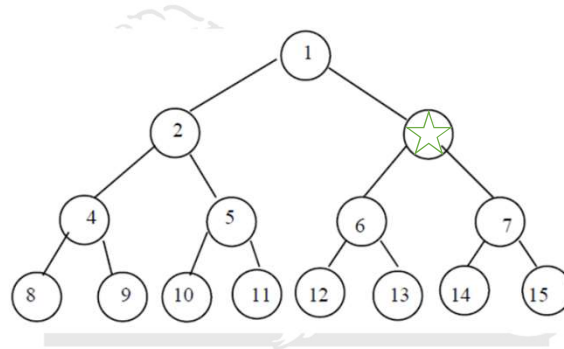
In this case 8 quorums are formed from 8 possible root-leaf paths:

1-2-4-8, 1-2-4-9, 1-2-5-10, 1-2-5-11, 1-3-6-12, 1-3-6-13, 1-3-7-14 and 1-3-7-15.

Handling Faults



- If any site fails, the algorithm substitutes for that site two possible paths starting from the site's two children and ending in leaf nodes.



- When node 3 fails

The algorithm for constructing a tree quorum



```

FUNCTION GetQuorum (Tree: NetworkHierarchy): QuorumSet;
  VAR left, right : QuorumSet;
  BEGIN
    IF Empty (Tree) THEN
      RETURN ({});
    ELSE IF GrantsPermission(Tree↑.Node) THEN
      RETURN ((Tree↑.Node) ∪ GetQuorum (Tree↑.LeftChild));
    OR
      RETURN ((Tree↑.Node) ∪ GetQuorum (Tree↑.RightChild));(*line 9*)
    ELSE
      left ← GetQuorum(Tree↑.left);
      right ← GetQuorum(Tree↑.right);
      IF (left = ∅ ∨ right = ∅) THEN
        (* Unsuccessful in establishing a quorum *)
        EXIT(-1);
      ELSE
        RETURN (left ∪ right);
      END; (* IF *)
    END; (* IF *)
  END; (* IF *)
END GetQuorum
  
```

When the number of node failures is greater than or equal to $(\log n)$, the algorithm may not be able to form tree-structured quorum

Mutual Exclusion Algorithm

A site s enters the critical section (CS) as follows:

1. Site s sends a 'Request' message to all other sites in the structured quorum it belongs to.
2. Each site in the quorum stores incoming requests in a request queue, ordered by their timestamps.
3. A site sends a 'Reply' message, indicating its consent to enter CS, only to the request at the head of its request queue, having the lowest timestamp.
4. If the site s gets a 'Reply' message from all sites in the structured quorum it belongs to, it enters the CS.

Mutual Exclusion Algorithm

5. After exiting the CS, s sends a **'Relinquish'** message to all sites in the structured quorum.
6. On the receipt of the 'Relinquish' message, each site removes s 's request from the head of its request queue.
7. If a new request arrives with a timestamp smaller than the request at the head of the queue
 - an 'Inquire' message is sent to the process whose request is at the head of the queue and
 - waits for a 'Yield' or 'Relinquish' message.

8. When a site s receives an **'Inquire' message**, it acts as follows:
 - If s has acquired all of its necessary replies to access the CS, then it simply **ignores** the 'Inquire' message and proceeds normally and sends a 'Relinquish' message after exiting the CS.
 - If s has not yet collected enough replies from its quorum, then it sends a 'Yield' message to the inquiring site.
9. When a site gets the **'Yield' message**, it puts the pending request (on behalf of which the 'Inquire' message was sent) at the head of the queue and sends a 'Reply' message to the requestor.

Correctness proof

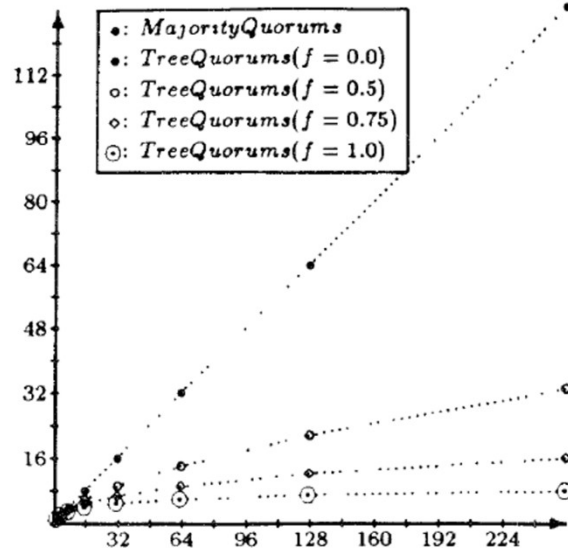
- Mutual exclusion is guaranteed because the set of quorums satisfy the Intersection property.
- Assumes FIFO channels
- Quorum Size
 - Best case $O(\log n)$
 - Worst case $O((n+1)/2)$
- In a network of 127 sites, the expected size of the quorum
 - In the majority quorum algorithm is 64
 - In the tree quorum algorithm will have size 7 in the best case and 22 when $f = 0.5$.

Message Complexity

- Number of messages needed to construct a quorum depend on Quorum Size
- Let C_l be the average cost of forming a quorum in a tree of level l
- Let f indicate the fraction of quorums that include the root of a tree of level $l + 1$

$$C_{l+1} = f(C_l + 1) + (1 - f)(2C_l).$$

Message Complexity



IIT ROORKEE

31

THEOREM:

The worst-case tree quorum size is ceiling $((n+1)/2)$

(proof by induction)

- Consider a binary tree of level 0, i.e., a tree consisting of a single site.
 - The size of the quorum is one and therefore the theorem holds.
- Assume that the theorem holds for binary trees of level k , i.e., the worst case quorum size is 2^k since there are $2^{k+1} - 1$ sites

- Consider a binary tree of level $k + 1$.
 - This tree consists of a root and left and right subtrees each of level k .
 - The largest quorum in such a tree will occur when the root is down and the largest quorums are chosen from the left and right subtrees.
 - The worst case size of the quorum is $2^k + 2^k$ or 2^{k+1} , which is equal to ceiling $[(n + 1)/2]$.
 - $n = 2^{k+2} - 1$.

Agarwal-El Abbadi Quorum-Based Algorithm



- When each node has degree d
- Any set of sites in a path from root to a leaf form a quorum
- whenever a node is inaccessible, the set must contain paths starting from all d children to the leaves
- size of a tree quorum in the best case ceiling $(\log_d n)$
- In worst case ceiling $((d-1)n + 1)/d$
- if the maximum number of possible failures is t then the tree must be of level t .

```

FUNCTION GetQuorum(Tree : TREE; Requestor : SITE) : Quorum Set;
VAR
  ChildQuorum : ARRAY[1..MaxDegree] OF QuorumSet;
BEGIN
  IF Empty(Tree) THEN
    RETURN({});
  ELSE IF GrantsPermission(Tree ↑ .NODE) THEN
    Pick I such that either:
    • Requestor is in the subtree rooted at Tree ↑ .Child[I], or
    • I is a random number in [1..Tree ↑ .Degree];
    RETURN({ Tree ↑ .Node } ∪ GetQuorum(Tree ↑ .Child[I], Requestor));
  ELSE
    FOR i ∈ [1..Tree ↑ .Degree] : ChildQuorum[i] ← GetQuorum(Tree ↑ .Child[i], Requestor);
    IF (∃ i ∈ [1..Tree ↑ .Degree] : ChildQuorum[i] = ∅) THEN
      (* Unsuccessful in establishing a quorum *)
      EXIT(error);
    ELSE
      RETURN( $\bigcup_{i=1}^{tree \uparrow Degree} ChildQuorum[i]$ );
    END; (* IF *)
  END; (* IF *)
END GetQuorum;

```

Fig. 7. The algorithm for constructing a quorum on an arbitrary tree.